



Phase 6

The Merchant

Layer

OmniERP — Design system, UX transformation, command palette,
invoice composer, dashboard, POS, onboarding, Excel import, PWA

Extract, don't transcribe — 8–9 weeks

DOCUMENT	Phase 6 Specification
VERSION	1.0
DATE	26 July 2026
AUTHOR	Principal Architecture & Product
AUDIENCE	Principal Systems Architect & coding assistant
STATUS	7 open questions (Q-32 to Q-38)

0. The Stitch Reality Check

You have design assets. You do not yet have a design system. The distinction determines whether this phase produces a maintainable frontend or a liability.

WHAT STITCH PRODUCES	WHAT PHASE 6 NEEDS
Rendered screens with baked-in values (<code>#2563EB</code> , <code>padding: 14px</code>)	A token layer that every screen references and nothing hardcodes
~5 rows of plausible fake data	Virtualized tables against 50,000 real rows
One state per screen — the happy one	Five states per screen: loading, empty, error, permission-denied, filtered-to-nothing
Almost certainly LTR	Arabic-first RTL as the default case
Screens as the unit of delivery	Primitives as the unit of delivery; screens are composition
No permission awareness	Every control gated by RBAC and branch scope

The governing rule for this phase: extract, don't transcribe.

Stitch is your **visual reference and aesthetic authority** — palette, type scale, spacing rhythm, component look. It is not your implementation source. The coding assistant should read a Stitch screen to answer *"what should this look like?"* and read the token file and primitive library to answer *"what do I build with?"*

Concretely: no screen in this codebase may contain a colour literal, a pixel literal, or a font-family declaration. If a value is needed that isn't in the token set, the correct action is to add a token, not to inline the value. Enforce it in CI (§3.3).

0.1 Token Reconciliation — Do This in Week 1

SRS v1.0 §8.2 already specifies a token set. Stitch has its own. They will conflict. Resolve it once, deliberately, before any component is built.

Stitch wins on aesthetics. Palette, type scale, spacing rhythm, radius, shadow treatment, component silhouette. You hired it for a point of view; use it.

The SRS wins on these six, non-negotiably, because they are correctness requirements wearing design clothing:

#	REQUIREMENT	WHY IT ISN'T NEGOTIABLE
R-1	Tabular figures (<code>font-variant-numeric: tabular-nums</code>) on every number in every table and every total	Non-aligned digits in a financial column read as broken software
R-2	Latin numerals (0–9) by default in financial contexts , with an Arabic-Indic toggle	Egyptian merchants read financial figures in Latin numerals; Arabic-Indic digits in a ledger get read as an error
R-3	RTL-first via logical properties	The product's primary language

#	REQUIREMENT	WHY IT ISN'T NEGOTIABLE
R-4	Semantic colour tokens including a four-token Trust Badge risk scale	Colour must carry meaning, and the meaning must survive a palette change
R-5	Light mode is the default	Accounting work happens in light mode; dark is fully supported but not the default
R-6	Density: 36px comfortable / 30px compact table rows, user-toggleable	Data-entry clerks are the highest-frequency users

If Stitch's chosen body face lacks Arabic coverage or tabular figures, it is replaced for data contexts and kept for display contexts. Decide this in week 1, not week 6.

1. Verification Gates

Blueprint II scheduled a frontend track during Phase 0 weeks 1–7: design tokens, RTL primitives, the DataTable, the command palette shell, and the Arabic PDF pipeline. Phase 5 then required the Arabic PDF pipeline for FR-576 (self-rendered receipts), so at least part of it must exist.

#	CONFIRM	IF FALSE
V-1	Arabic PDF pipeline with correct shaping, bidi, and font embedding exists (built for Phase 5 FR-576)	+1 week, and it blocks POS receipt printing
V-2	<code><DataTable/></code> with server-side pagination, sorting, filtering, and virtualization exists	+2 weeks. This is the largest single component in the phase
V-3	Command palette shell exists	+1 week
V-4	ESLint rule banning <code>left / right</code> physical properties is active	+2 days, and every component built so far needs auditing
V-5	A typed API client is generated from the OpenAPI spec, not hand-written	+1 week. Hand-written fetch calls against a 5-phase backend will drift
V-6	Performance budgets (bundle size, Lighthouse) are active CI gates	+2 days

Answer V-1 through V-6 before committing to the 8–9 week estimate. If all six are false, this phase is 12 weeks.

2. Architectural Goals

- Tokens are the single source of truth.** One file generates CSS variables, the Tailwind theme, and typed TS exports. Changing a brand colour is a one-line diff.
- Dependencies flow one direction.** Primitives never know about domain concepts. Screens never contain styling logic.

3. **RTL is the default case, LTR is the variant.** Not the reverse.
4. **Server state and client state never mix.** React Query owns anything fetchable; Zustand owns only ephemeral UI state.
5. **Every screen ships five states.** A screen with only its happy path is not done.
6. **Money and quantity are types, not numbers.** Formatting is encapsulated so no screen can render a figure wrongly.
7. **The POS works with no network, and tells the truth about what that costs.** Especially now that Phase 5's 72-hour ETA window is live.
8. **Performance budgets are CI gates, not aspirations.**
9. **Permission and branch scope are enforced in the component layer**, mirroring the query-layer enforcement from Phase 1c.

3. Frontend Architecture

3.1 The Layer Cake

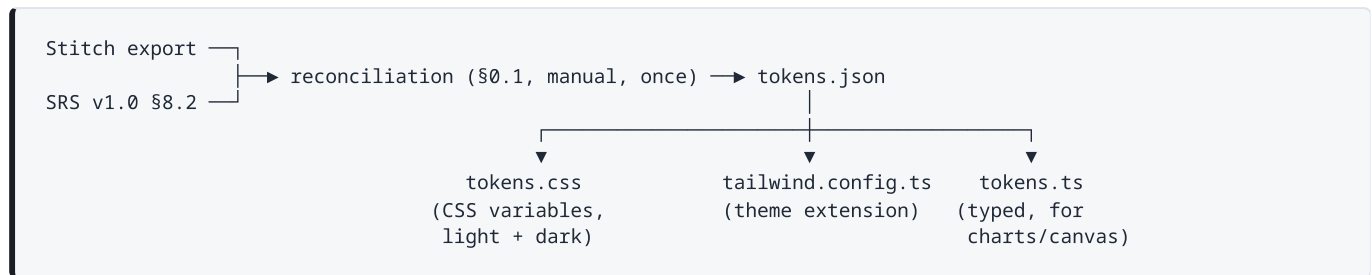
```

src/
├── tokens/                                generated — no React, no imports
│   ├── tokens.json                       canonical source (reconciled Stitch + SRS)
│   ├── tokens.css                        CSS custom properties, light + dark
│   ├── tokens.ts                         typed exports for JS-land (charts, canvas)
│   └── build.ts                          generator; runs in CI
├── primitives/                           zero domain knowledge
│   ├── Button Input Select Combobox Checkbox Radio Switch Textarea
│   ├── Badge Card Dialog Drawer Popover Tooltip Tabs Toast
│   └── Skeleton Spinner Avatar Separator ScrollArea
├── patterns/                             composed, still domain-agnostic
│   ├── DataTable FilterBuilder SavedViews PageHeader FormLayout
│   ├── EmptyState ErrorState PermissionDenied StateBoundary
│   └── CommandPalette WidgetGrid Stepper FileDropzone
├── domain/                               OmniERP concepts — the important layer
│   ├── MoneyText QtyText DateText        formatting correctness
│   ├── TrustBadge EtaStatusBadge         Phase 5 + Trust Network
│   ├── DocumentStateBadge                Phase 4 approval states
│   ├── ContactPicker ItemPicker WarehousePicker BranchPicker
│   ├── TaxCodeSelect CostLayerInspector  Phase 1a
│   └── ApprovalInboxCard                 Phase 4a
├── features/                             screen-level composition
│   ├── invoice-composer/ pos/ onboarding/ import/ dashboard/
│   ├── contacts/ inventory/ purchases/ accounting/ eta/
└── app/                                  Next.js routes, layouts, providers

```

Enforced rule (FR-601): imports flow downward only. `primitives/` may not import from `patterns/`, `domain/`, or `features/`. `patterns/` may not import from `domain/` or `features/`. Enforce with `eslint-plugin-boundaries` or `import/no-restricted-paths`, as a **blocking** CI check — the same discipline that kept your backend bounded contexts clean.

3.2 The Token Pipeline



Token categories, minimum viable set:

```

color.bg.{base,subtle,muted,inverse}
color.fg.{base,muted,subtle,inverse,onAccent}
color.border.{base,subtle,strong,focus}
color.accent.{base,hover,active,subtle}
color.semantic.{positive,negative,warning,info}[.base|.subtle|.fg]
color.risk.{unknown,good,caution,high}           - Trust Badge scale
color.docstate.{draft,pending,approved,posted,rejected,reversed}
color.eta.{queued,submitted,accepted,rejected,atRisk}

font.family.{display,body,data,mono}
font.size.{2xs,xs,sm,base,lg,xl,2xl,3xl}
font.weight.{regular,medium,semibold,bold}
font.numeric.tabular                               - R-1

space.{0,1,2,3,4,5,6,8,10,12,16,20,24}             4px base
radius.{none,sm,base,lg,full}
shadow.{none,popover}                             borders over shadows
motion.duration.{fast,base,slow}                  120/160/220ms
motion.easing.{out,inOut}

density.row.{comfortable,compact}                 36px / 30px
z.{base,sticky,dropdown,dialog,toast}
  
```

Dark mode is a token swap, never a second stylesheet. `[data-theme="dark"]` redefines the same variable names.

3.3 CI Gates for the Design System

GATE	CHECK
no-raw-color	No hex, <code>rgb()</code> , or <code>hsl()</code> literals outside <code>tokens/</code>
no-magic-spacing	No <code>px</code> literals in <code>className</code> or styled blocks outside <code>tokens/</code>
no-physical-props	No <code>left</code> / <code>right</code> / <code>margin-left</code> / <code>padding-right</code> / <code>text-align: left</code> — logical properties only
import-boundaries	Layer cake dependency direction
no-raw-money	No <code>.toFixed()</code> or <code>Intl.NumberFormat</code> outside <code>domain/MoneyText</code>
bundle-size	Core route < 180KB gz; per-feature chunk < 90KB gz
lighthouse-ci	LCP < 2.5s on simulated 3G Fast; INP < 200ms; CLS < 0.05
ally	axe-core on every route: zero critical violations

3.4 State Management Boundary

The rule: if it can be re-fetched from the server, React Query owns it. Zustand owns nothing that exists in the database.

REACT QUERY (SERVER STATE)	ZUSTAND (CLIENT STATE)
Every entity list and detail	Sidebar collapsed/expanded
Dashboard widget data	Command palette open + query
Trust scores, ETA status	Active POS shift buffer (pre-commit)
Saved views, dashboard layouts	Offline POS queue (mirrored to IndexedDB)
Permissions, tenant config	Theme, density, language preference
	Invoice composer draft (pre-save)
	Import wizard step + column mapping

Anti-pattern to ban explicitly: copying fetched server data into a Zustand store. It creates two sources of truth and a class of staleness bugs that are painful to trace.

3.5 The API Contract Layer

FR-602 (P0): the typed client is **generated** from the backend's OpenAPI spec (`openapi-typescript` + a query-hook generator), committed, and regenerated in CI. A drift between spec and client fails the build.

You have five phases of backend surface — cost layers, approvals, procure-to-pay, ETA, GL. Hand-written fetch calls against that will diverge silently and you will find out in production.

4. RTL Strategy

Stitch output is LTR. This is systematic work; do it once at the token/primitive layer rather than forty times at the screen layer.

ID	REQUIREMENT
FR-610	All layout uses logical properties: <code>margin-inline-start</code> , <code>padding-inline-end</code> , <code>inset-inline-start</code> , <code>border-inline-start</code> , <code>text-align: start</code> . Tailwind's <code>ms-</code> / <code>me-</code> / <code>ps-</code> / <code>pe-</code> / <code>start-</code> / <code>end-</code> utilities, never <code>ml-</code> / <code>mr-</code> / <code>pl-</code> / <code>pr-</code> / <code>left-</code> / <code>right-</code>
FR-611	Stitch conversion codemod: a one-time script mapping physical utilities to logical equivalents across the imported markup, plus the ESLint rule to prevent regression
FR-612	Direction-aware icon mirroring. Arrows, chevrons, undo/redo, and progress indicators mirror; search, user, calendar, and settings do not. Implement as an <code><Icon mirror></code> prop, not per-usage CSS
FR-613	<bdi> isolation for mixed Arabic/Latin strings — SKUs, invoice numbers, tax IDs, serial numbers, and EGS codes inside Arabic sentences. Without it, an invoice number renders in visually scrambled order

ID	REQUIREMENT
FR-614	Charts mirror under RTL: value axis on the right, time flowing right-to-left
FR-615	Language toggle is instant, no reload, persisted per user, and re-renders <code>dir</code> at the document root
FR-616	Arabic numeral toggle (R-2) applies to display only — never to input parsing, never to stored values
FR-617	Every primitive has an RTL visual regression snapshot in CI

5. Functional Requirements

5.1 DataTable — the component used on ~30 screens

ID	REQUIREMENT
FR-620	Server-side pagination (cursor-based), sorting, filtering, and search. Never fetch a full result set to the client
FR-621	Row virtualization beyond 100 visible rows (TanStack Virtual)
FR-622	Column show/hide, reorder, resize, pin — persisted per user per table
FR-623	Filter builder: field + operator + value, AND/OR groups, saved as named views, shareable within the tenant
FR-624	Multi-select with select-all-matching-filter (not just the visible page), clear count, confirmation on destructive actions
FR-625	Inline editing on permitted cells with optimistic update and rollback on failure
FR-626	Sticky header, sticky totals row, sticky first column
FR-627	Keyboard navigation across cells; <code>Enter</code> opens the record; <code>J / K</code> move rows; <code>X</code> selects
FR-628	Export respecting current filters and column selection
FR-629	Density toggle, persisted
FR-630	Skeleton rows matching final row height exactly — zero cumulative layout shift
FR-631	All five states (§5.9), including filtered-to-nothing with a clear-filters action
FR-632	Bulk actions bar appears on selection: bulk print, bulk ETA submit, bulk approve, bulk payment

5.2 Command Palette

ID	REQUIREMENT
FR-640	<code>⌘K</code> / <code>Ctrl+K</code> . Opens in < 50ms , local-first, server results stream in
FR-641	Unified results: navigation · records across all entities · actions ("فاتورة جديدة", "سند قبض") · recent items · settings

ID	REQUIREMENT
FR-642	Scanned barcode or serial number resolves to its record — the scanner types fast into the palette and the palette recognises the burst pattern
FR-643	Actions are contributed by a registry, so plugins (and Phase 8's Copilot) extend it without touching Core
FR-644	Permission-filtered — a user never sees an action they cannot perform
FR-645	Global keyboard map: <code>G</code> then <code>D/S/I/C/P</code> go-to · <code>N</code> new (context-sensitive) · <code>/</code> search · <code>⌘Enter</code> save · <code>Esc</code> cancel · <code>?</code> shortcut sheet

5.3 Invoice Composer — the demo screen

This is the screen the whole design serves (SRS v1.0 §8.7).

ID	REQUIREMENT
FR-650	Single screen. A three-line invoice completed in under 20 seconds, keyboard only. No modals, no page transitions, no wizard. This is a measured, tested benchmark, not an aspiration
FR-651	Customer autocomplete by name or phone; item autocomplete by name, SKU, or barcode; inline quantity and price edit; live totals
FR-652	Trust Badge resolves inline on phone entry , p95 < 150ms, with Arabic reason text
FR-653	Serial and batch selection inline for tracked items, validated against reservation
FR-654	ETA readiness indicator before save — missing EGS code or tax mapping is surfaced here, not after rejection
FR-655	Approval preview: "هتحتاج موافقة المدير" shown before submit when a Phase 4 rule will match
FR-656	Draft autosave to Zustand + localStorage; survives an accidental navigation
FR-657	<code>Ctrl+Enter</code> saves; the next invoice opens with the cursor in the customer field

5.4 Dashboard & Widget Contribution

ID	REQUIREMENT
FR-660	Core owns the responsive grid, layout persistence, global time-range control, and refresh orchestration
FR-661	Plugins register widget descriptors: <code>{id, titleAr, titleEn, size:{w,h,minW,minH}, dataKey, component, permissions, refreshStrategy, drillThroughRoute}</code>
FR-662	Each widget renders inside an Error Boundary and a Suspense boundary. A widget that throws or hangs degrades to a placeholder tile; it never breaks the dashboard
FR-663	Single batched data call — <code>POST /dashboard/resolve</code> with a list of widget ids and params. Twelve widgets must not mean twelve round trips
FR-664	Drag, resize, add, remove; layout persisted per user per tenant; industry presets provide the starting layout
FR-665	Every widget drills through to a filtered list view

ID	REQUIREMENT
FR-666	Launch widget set: Cash Position · Receivables Aging · Payables Due · Sales Trend · Gross Margin · Top Items · Stock Alerts · ETA Health · Approval Inbox · GRNI Aging · Trust Network Savings · COD Funnel
FR-667	Dashboard full render p95 < 1.2s with 12 widgets, on 3G, with 100k transactions in the tenant

5.5 POS

Now materially more complex than the original spec, because Phase 5 is live.

ID	REQUIREMENT
FR-670	Full-screen, touch-optimised, large hit targets, one-handed reachable
FR-671	Offline-capable : IndexedDB queue, deterministic sync on reconnect, no data loss
FR-672	Shift management: open with float, blind close (the counter does not see the expected figure), variance posting to the GL
FR-673	Barcode scanning via keyboard-wedge and PWA camera
FR-674	Split payment across methods; cash rounding; change calculation
FR-675	Park and retrieve sale; multiple concurrent open sales
FR-676	e-Receipt integration (Phase 5) : receipt enters the batch queue on issue; <code>date_time_issued</code> starts the 72-hour window at <i>creation</i> , not at sync
FR-677	Offline duration warning — the cashier is told when offline time approaches the ETA window: "غير متصل من 40 ساعة — لازم اتصال بالإنترنت خلال 32 ساعة" (Phase 5 FR-574)
FR-678	Receipts syncing after their window has expired are flagged <code>late_on_arrival</code> , submitted anyway, and surfaced in the ETA Health widget
FR-679	Thermal (80mm) and A4 receipt printing with verification QR, using the local PDF pipeline (Phase 5 FR-576)
FR-680	Terminal registration and per-terminal reporting

5.6 Onboarding

ID	REQUIREMENT
FR-690	First real invoice within 10 minutes of signup . The measured target for the whole flow
FR-691	Business type selection (E-commerce / Wholesale / Retail / Services) auto-installs the plugin bundle, dashboard preset, sidebar order, and default categories
FR-692	Company profile: name, tax registration, address, logo, base currency, fiscal year start
FR-693	Resumable and skippable, with progress persisted server-side. A merchant who abandons at step 3 returns to step 3
FR-694	Opening balances: cash, bank, customer balances, supplier balances, stock on hand — producing a balanced opening journal entry
FR-695	ETA setup deferred to its own guided flow (Phase 5 FR-584), not blocking first invoice

ID	REQUIREMENT
FR-696	Sample-data mode for evaluation, with one-click purge before going live
FR-697	Empty states across the product are onboarding surfaces — each teaches the next action rather than reporting absence

5.7 Excel / CSV Import

The highest-leverage adoption feature in the product (SRS v1.0 FR-105). Merchants have their history in spreadsheets; without import, they do not switch.

ID	REQUIREMENT
FR-700	Supported entities: Contacts · Items (with variants) · Opening stock with costs · Opening balances · Price lists · EGS codes · Historical COD outcomes (Trust Network backfill)
FR-701	Column mapping UI with auto-detection by header name (Arabic and English), remembered per tenant per entity
FR-702	Dry-run validation preview before commit: per-row pass/fail, error reason, and a downloadable error report
FR-703	Partial commit — valid rows import, invalid rows are returned for correction and re-upload. All-or-nothing on a 4,000-row file is user-hostile
FR-704	Processing as an ARQ <code>bulk</code> job with progress reporting; a 20,000-row file must not block the UI or a request thread
FR-705	Idempotency: re-uploading the same file does not duplicate. Match on a natural key or an import batch id
FR-706	Opening stock import creates real cost layers via the Phase 1a engine — not a direct quantity write. Costing method, unit cost, warehouse, batch, and serial all supported
FR-707	Downloadable templates per entity, Arabic headers, with an example row
FR-708	Import history with rollback for a batch that has not yet been transacted against
FR-709	Trust Network historical backfill — importing past COD outcomes seeds the network on day one (SRS v2.0 R-10 cold-start mitigation)

5.8 PWA & Offline

ID	REQUIREMENT
FR-710	Installable PWA with correct manifest, icons, splash, and Arabic name
FR-711	Service worker: app shell precached, stale-while-revalidate for reference data, network-first for transactional data
FR-712	Offline scope is explicit and narrow : POS sales, and read-only access to recently viewed records. Everything else shows an honest offline state
FR-713	Sync status is always visible — never a silent queue
FR-714	Push notifications for approvals (Phase 4 FR-1214) and ETA at-risk alerts
FR-715	Works usably on 3G with 400ms RTT

5.9 The Five States

FR-720 (P0): every data-bearing screen implements all five. A screen with only its happy path does not pass review.

STATE	REQUIREMENT
Loading	Skeleton matching the final layout's dimensions exactly. Zero layout shift
Empty	Teaches the next action. "لسه مفيش فواتير — اعمل أول فاتورة" with the action attached. Never a bare "No data"
Error	States what went wrong and offers a retry. Never a raw error code, never an apology, never vague
Permission denied	Explains that access is restricted and who to ask. Not a 403 page, not a blank screen
Filtered to nothing	Distinguished from genuinely empty, with a clear-filters action

Copy discipline: active voice, plain verbs, sentence case. A control names exactly what happens — "احفظ الفاتورة", not "إرسال". The action keeps the same name through the whole flow, so the button that says "اعتماد" produces a toast that says "تم الاعتماد".

6. Domain Entities

Phase 6 is mostly frontend, but it needs backend support. New tables in the tenant schema unless noted:

```

user_preferences
  user_id, theme, density, language, numeral_system,
  default_branch_id, default_warehouse_id, sidebar_collapsed

dashboard_layouts
  user_id, preset_id, layout jsonb  -- [{widgetId, x, y, w, h}]

saved_views
  user_id?, entity_type, name, filters jsonb, columns jsonb,
  sort jsonb, is_shared, created_by

import_jobs
  id, entity_type, filename, storage_ref, state,
  total_rows, valid_rows, imported_rows, failed_rows,
  mapping jsonb, errors jsonb, batch_id, created_by

import_mappings  -- remembered column mappings
  tenant_id, entity_type, source_header, target_field, confidence

onboarding_progress
  tenant_id, current_step, completed_steps[], skipped_steps[],
  industry_preset, is_complete

industry_presets  -- public schema, platform-managed
  code, name_ar, name_en, plugins[], dashboard_layout jsonb,
  sidebar_order[], default_categories[], default_sequences jsonb

pos_terminals
  id, branch_id, name, device_fingerprint,
  last_seen_at, last_sync_at, eta_terminal_ref

pos_shifts
  id, terminal_id, user_id, opened_at, opening_float,
  closed_at, counted_cash, expected_cash, variance,
  variance_journal_entry_id

pos_offline_queue  -- client-side IndexedDB, mirrored
  local_id, terminal_id, payload, created_at,
  sync_state, server_id, conflict_reason

```

Frontend-only state (Zustand, never persisted server-side): command palette, invoice composer draft, import wizard step, active shift buffer, offline queue mirror.

7. The Hard Problems

7.1 Offline POS Stock Conflicts

Two terminals go offline. Both sell the last unit of a SKU. Both sync.

There is no clever resolution — one sale is real and one is a problem. Design honestly:

1. POS holds a **local optimistic reservation**; the server is always authoritative.
2. On sync, the server applies movements in `occurred_at` order through the Phase 1a engine.
3. The losing sale creates a **negative-stock provisional layer** (Phase 1a FR-331) rather than being rejected — the goods physically left the shop; the ledger must reflect that.
4. The conflict surfaces as a **reconciliation task**, not a silent overwrite: "بيعت 1 قطعة أكثر من الرصيد على تيرمينال 2".
5. The merchant resolves by receiving stock or posting an adjustment.

Reject any design that discards a sale on sync. The money was taken; the ledger must agree with reality.

7.2 The 20-Second Invoice

The benchmark is a real engineering constraint, and most of it is network latency, not rendering.

- Contact and item search must be **local-first** — prefetch the top 500 contacts and top 1,000 items into an in-memory index on app load, then merge server results as they stream.
- Trust Badge fires on phone-field blur, in parallel with everything else, and never blocks save.
- Save is **optimistic** — the invoice appears posted immediately, with rollback and a clear toast on failure.
- ETA submission is fire-and-forget into the Phase 5 queue.

Instrument it. Ship a hidden timing harness that measures composer-open → saved, and put the p95 on your internal dashboard. A benchmark nobody measures decays within a month.

7.3 Data Density Versus Mobile

The command-centre direction (dense tables, keyboard-driven) and the PWA-first direction pull opposite ways. Resolve it by **surface**, not by compromise:

SURFACE	DIRECTION
Desktop lists, accounting, reports	Dense, keyboard, command-centre
POS	Touch-first, large targets, no dense tables
Approvals	Mobile-first — approvers are not at a desk
Dashboard	Responsive; widgets reflow to a single column
Composer	Desktop-optimised, mobile-usable

Do not build one responsive table that tries to be both. On mobile, list views switch to a card layout — a different component behind the same route, not a squeezed table.

7.4 Permission-Aware Rendering at Scale

You have granular RBAC (`cost.view` separable from `invoice.create`), branch scoping, and Phase 4 approval permissions. Rendering that correctly across ~40 screens without scattering conditionals everywhere:

FR-730: a single `<Can permission="cost.view" branch={branchId}>` primitive plus a `usePermission()` hook, both reading from a permission set fetched once and cached. **No screen performs its own role comparison.** A sales user opening an item detail must not see cost — and that must be enforced by the API as well, since hiding a field in the UI is not a security control.

7.5 Stitch Screens Without States

Your Stitch folder has happy-path screens. The five states (\$5.9) do not exist in it, and they are more than half the real work.

Process requirement: for each Stitch screen the coding assistant implements, it produces the happy path *and* the five states in the same PR. Reviewing them separately guarantees the states never get built.

8. Suggested Sequencing (8–9 weeks)

WEEK	WORK
1	Token reconciliation (\$0.1) · token build pipeline · CI gates · generated API client · Stitch RTL codemod
2	Primitives layer, complete, with RTL visual regression snapshots
3	Patterns: DataTable core, StateBoundary, FilterBuilder, PageHeader, FormLayout
4	DataTable advanced (saved views, bulk actions, virtualization) · command palette
5	Domain components · dashboard grid + widget contract + launch widget set
6	Invoice composer + the 20-second benchmark harness
7	POS: shifts, offline queue, sync, e-Receipt integration, printing
8	Onboarding wizard · industry presets · Excel import
9	PWA · performance tuning to budget · accessibility pass · buffer

Week 1 is the gate. If tokens are not reconciled and generating by end of week 1, every component built afterwards accumulates a hardcoded value that has to be found and removed later.

9. Exit Criteria

- ✓ Zero colour, spacing, or font literals outside `tokens/` — CI-enforced
- ✓ Zero physical direction properties — CI-enforced
- ✓ Import boundaries hold; no primitive imports a domain component
- ✓ Every primitive has light, dark, LTR, and RTL visual regression snapshots
- ✓ **Three-line invoice created in under 20 seconds, keyboard only**, measured on the timing harness at p95
- ✓ Dashboard renders 12 widgets in **< 1.2s p95** on 3G with 100k transactions
- ✓ A deliberately throwing widget degrades to a placeholder without breaking the dashboard
- ✓ DataTable handles 50,000 rows with server-side paging and virtualization, sort/filter response < 400ms p95
- ✓ Command palette opens in < 50ms; a scanned serial resolves to its record
- ✓ **POS completes 20 sales fully offline, syncs cleanly, and correctly flags a receipt whose 72-hour ETA window expired while offline**
- ✓ Two terminals selling the last unit offline produce one reconciliation task, zero lost sales, and a correct provisional cost layer
- ✓ Signup → first posted invoice in **under 10 minutes**, timed with a real person who has not seen the product
- ✓ 4,000-row item import with 200 deliberately bad rows: valid rows commit, bad rows return with per-row reasons, re-upload does not duplicate
- ✓ Opening stock import produces cost layers reconciling to the inventory GL account
- ✓ Every data screen ships all five states — verified by review checklist, not by intention

16. ☒ Bundle, Lighthouse, and axe budgets green in CI
17. ☒ Arabic PDF/print verified on a physical thermal printer and A4
18. ☒ A sales-role user cannot see cost anywhere in the UI, and cannot retrieve it from the API either

10. Open Questions

#	QUESTION	URGENCY
Q-32	What is actually in the Stitch folder? Screenshots only, or HTML/CSS, or Figma with named styles? This changes week 1 substantially — extractable tokens versus eyeballed values from PNGs	Blocking week 1
Q-33	Answer V-1 through V-6 (§1). If the Phase 0 frontend track did not happen, this phase is 12 weeks, not 8–9	Blocking the estimate
Q-34	Does the Stitch design cover POS and mobile, or desktop screens only? POS is a different design language and may need its own direction	Week 1
Q-35	Do Stitch's chosen typefaces have full Arabic coverage and tabular figures? If not, a data face is substituted (§0.1)	Week 1
Q-36	Do you have a real merchant's messy Excel file to test import against? Synthetic test files will not surface the problems real ones do	Week 8
Q-37	Which POS hardware — thermal printer model, barcode scanner, cash drawer? Printing needs testing on the real device, and 80mm Arabic thermal output is where surprises live	Week 7
Q-38	Third engineer status. This phase parallelises well: primitives/patterns, POS, and import are three near-independent tracks	Now

Q-32 and Q-33 both need answering before week 1 starts. They are the difference between an 8-week phase and a 12-week one, and it is better to know now.